

Sujet d'examen UE NFE204 - Session 1

Année universitaire 2020-2021

Examen session FOAD : 6 juillet 2021

Première partie : modélisation NoSQL (8 pts)

Dans le monde d'après, il n'y a plus de presse, de quotidiens ou de magazines. Des journalistes indépendants, payés au caractère, écrivent des articles et les proposent à des plateformes de diffusion. Ces journalistes sont (faiblement) rémunérés en fonction du nombre de personnes qui lisent l'article sur ces plateformes.

Posons-nous quelques questions sur le fonctionnement de cette presse ubérisée. Un modèle UML très sommaire est donné dans la figure 1. Les journalistes publient des articles, et ces articles sont diffusés par des plateformes (plusieurs plateformes si possible, qui elles-mêmes publient beaucoup d'articles).

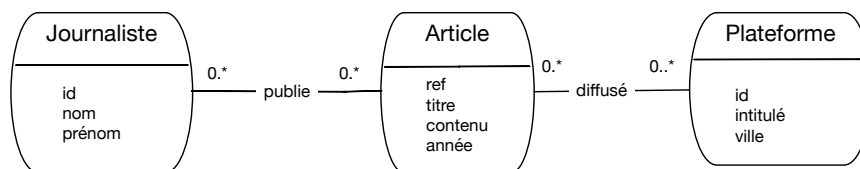


FIGURE 1 – Modèle (sommaire) de la diffusion d'articles ubérisés

Au départ, on représente tout cela avec une base relationnelle dont voici un tout petit échantillon.

ref	titre	année
ju21	Alerte aux gilets jaunes	2018
ju798	Aura-t-on des vaccins pour Noël	2021

La table *Article*

id	nom
nt	N. Travers
pr	P. Rigaux
rfs	R. Fournier-S'niehotta
cdm	C. du Mouza

La table *Journaliste*

refArticle	idAuteur
ju21	cdm
ju21	pr
ju21	nt
ju798	pr

La table des publications

idPlateforme	nom
pj2	Béru
pj5	Croco
pj6	Vautour

La table des plateformes

s refArticle	idPlateforme
ju21	pj2
ju798	pj6
ju21	pj6

La table des diffusions

1. Proposez un format de document JSON représentant toutes les informations présentes dans les tables ci-dessus et relatives à l'auteur *P. Rigaux* (représentation A).

Solution :

```
{ "_id": "pr",
  "nom": "P. Rigaux",
  "articles": [
    { "titre": "Alerte aux gilets jaunes",
      "année": 2018
      "plateformes": ["Béru", "Vautour"]
    },
    { "titre": "Aura-t-on des vaccins pour Noël",
      "année": 2021
      "plateformes": ["Vautour"],
    }
  ]
}
```

2. Proposez un document JSON représentant toutes les informations relatives à la plateforme *Vautour* présentes dans les tables ci-dessus (représentation B).

Solution :

```
{ "_id": "pj6",
  "nom": "Vautour",
  "articles": [
    { "titre": "Alerte aux gilets jaunes",
      "année": 2018
      "auteurs": ["N. Travers", "P. Rigaux", "R. Fournier-S'niehotta"]
    },
    { "titre": "Aura-t-on des vaccins pour Noël",
      "année": 2021
      "auteurs": ["P. Rigaux"],
    }
  ]
}
```

3. Expliquez le calcul MapReduce permettant de produire les documents de type B à partir des documents de type A.

Solution : Voici la modélisation en Javascript. Il faut, dans le Map, deux boucles imbriquées pour accéder aux plateformes dont le nom constitue la clé de regroupement dans la phase de Reduce.

```
function mapAuteurVersPlateforme (doc) {
  // Première boucle sur les articles
  for article in doc.articles {
    // Second boucle sur les plateformes
    for (plateforme in article.plateforme) {
      // La clé est donc le nom de la plateforme. On l'accompagne
      // d'un document contenant l'auteur, le titre et l'année
      emit (plateforme, {"auteur": doc.nom, "titre": article.titre,
                        "année": article.année})
    }
  }
}
```

Pour le Reduce : on reçoit, pour chaque plateforme, la liste des articles avec leurs auteurs. Il suffit de regrouper les articles par titre pour obtenir leurs auteurs.

```
function reduceAuteurVersPlateforme (plateforme, [listeArticles]) {
  // On va s'épargner le petit algo de regroupement des auteurs sur le titre
  les_articles = groupement_auteurs_par_titre (listeArticles)
  // On a tout ce qu'il faut
  return ({ "nom": plateforme, "articles": les_articles } )
}
```

Important : pour spécifier les calculs MapReduce, donner les fonctions de Map et de Reduce, en javascript, en pseudo-code, au pire en langage naturel en étant le plus précis possible.

4. On suppose que le contenu de la base est accessible à un processeur Pig latin sous forme de trois collections. La première collection est celle des articles avec des documents du type :

```
{ "ref" : "ju21", "titre": "Alerte aux gilets jaunes", "année": 2018 }
```

La collection des auteurs. Exemples :

```
{ "nom" : "P. Rigaux", "refArticle": "ju21" }
{ "nom" : "P. Rigaux", "refArticle": "ju798" }
```

Et enfin la collection des diffusions. Exemples :

```
{ "plateforme" : "Béru", "refArticle": "ju21" }
{ "plateforme" : "Vautour", "refArticle": "ju798" }
```

On veut obtenir, à partir de l'échantillon de base de données donné en début d'énoncé, un document représentant les informations sur un article :

```
{ "ref": "ju21",
  "titre" : "Alerte aux gilets jaunes",
  "année": 2018,
  "auteurs": [
    ],
  "plateformes": [
    ]
}
```

- Donnez le contenu JSON des champs auteurs et plateformes d'après la base relationnelle de l'énoncé.
- Quelle chaîne de traitement permet de passer des trois collections à la représentation sous forme de document ci-dessus ? Exprimez la chaîne de traitement avec des opérateurs (pas besoin d'une syntaxe complète, un texte précis ou un schéma commenté fait très bien l'affaire).

Solution : L'opérateur principal à mettre en œuvre est le cogroup. Le premier (mais l'ordre n'est pas important) groupe les articles et les auteurs.

```
articleEtAuteur = cogroup Articles by ref, Auteurs by refArticle;
```

Puis on regroupe la collection obtenue avec la collection des diffusions.

```
articlesComplet = cogroup articleEtAuteur by ref, Diffusions by refArticle;
```

- À votre avis combien faut-il d'opérations MapReduce successives pour effectuer ce calcul ?

Solution : Clairement, il en faut deux.

Deuxième partie : flux (4 pts)

Chaque fois que quelqu'un lit un article sur une plateforme, un message est produit. Le format de ce message est le suivant :

```
{"refArticle": "ju9", "contribution": 0.005}
```

La contribution est le montant (en Euros) généré par cette lecture. Ces messages arrivent en un flux nommé `lectures`. On veut écrire un traitement de flux qui cumule le montant des contributions par article.

1. En vous inspirant des chaînes de traitement sur les flux vues dans le dernier chapitre du cours, indiquez comment on maintient, pour chaque article, le cumul des contributions.

Solution : Voici la chaîne de traitement en Scala, décalquée à peu de choses près du compteur de mots.

```
case class Lecture(refArticle: String, contribution: float)
val w = lectures.keyBy("refArticle") // Regroupement par article
    .sum("contribution") // Cumul à chaque nouvelle lecture
    .print() // Optionnel: affiche l'évolution continue du cumul
senv.execute("Cumul des contributions")
```

2. On veut rémunérer les auteurs d'articles toutes les 6 heures. Quelle est la technique à utiliser pour obtenir le montant ?

Solution : Il faut évidemment faire un fenêtrage. On maintient un accumulateur `accum` dans lequel on ajoute les lectures reçues sur le flux, et ce pendant la durée de la fenêtre.

```
case class Lecture(refArticle: String, contribution: float)
val w = lectures.keyBy("refArticle") // Regroupement par article
    .window(6*60*60) // Si on exprime en secondes
    .reduce( (accum, lecture) => (accum.ref, accum + lecture.contribution) )
    .print()
senv.execute("")
```

3. Quelle est la parallélisation maximale permise par ce traitement ?

Solution : Il y a un sous-flux par article, ce qui permet un large parallélisme à priori.

Troisième partie : recherche d'information (6 pts)

Voici un échantillon du flux de nouvelles sur les élections régionales.

1. (d1) La Corse et l'Alsace sont deux régions pourvu d'une forte identité et d'une langue régionale. L'Alsace est plus calme...
2. (d2) L'Alsace, la Corse, et la Normandie n'ont pas grand chose à voir avec la Bretagne.
3. (d3) le nouveau président de la région Bretagne a prononcé son discours d'investiture.
4. (d4) La Corse et la Normandie montrent la progression du parti des chaussettes vertes : analyse détaillée pour la Normandie.

On veut indexer les nouvelles reçues de manière à pouvoir les rechercher en fonction d'une région. Le vocabulaire auquel on se restreint est donc celui des noms de pays (Alsace, Corse, Bretagne, Normandie).

1. Donnez une matrice d'incidence contenant les tf pour les quatre régions, et un tableau donnant les idf (sans appliquer le log). Mettez les noms de région en ligne, et les documents en colonne.

Solution :

	d1	d2	d3	d4
Alsace (2)	2	1	0	0
Corse (4/3)	1	1	0	1
Bretagne (2)	0	1	1	0
Normandie (2)	0	1	0	2

2. Donner les résultats classés par similarité cosinus basée sur les tf (on ignore l'idf) pour les requêtes suivantes. Expliquez brièvement le classement.

- Normandie ;
- Corse et Bretagne.

Solution : Les normes

— $\|d_1\| = \sqrt{4+1} = \sqrt{5}$; $\|d_2\| = \sqrt{1+1+1+1} = 2$; $\|d_3\| = \sqrt{1}$; $\|d_4\| = \sqrt{1+4} = \sqrt{5}$.

Les cosinus (requête non normalisée).

— Normandie : d2 : $\frac{1}{2} = 0,5$; d4 : $\frac{2}{\sqrt{5}} \approx 0,89$

d4 est premier car il mentionne deux fois la Normandie.

— Alsace et Corse

d1 : $\frac{2+1}{\sqrt{5}}$; d2 : $\frac{1+1}{2}$; d4 : $\frac{1}{\sqrt{5}}$;

d1 est premier comme on pouvait s'y attendre : il parle exclusivement de la Corse et de l'Alsace. Viennent ensuite d2 puis d4.

— Corse et Bretagne.

d1 : $\frac{1}{\sqrt{5}}$; d2 : $\frac{2}{2}$; d3 : $\frac{1}{1}$; d4 : $\frac{1}{\sqrt{5}}$;

d2 et d3 arrivent à égalité. Intuitivement, ils parlent tous les deux "à moitié" de Corse et de Bretagne. d1 et d4 parlent de Corse *ou* de Bretagne et aussi des autres pays.

3. Reprenons la dernière requête, "Corse et Bretagne" et les documents d2 et d3. Qu'est-ce que la prise en compte de l'idf changerait au classement ?

Solution : La Bretagne a un idf plus élevé : d3 serait donc considéré comme plus pertinent, car plus spécifique au besoin exprimé par la requête.

Quatrième partie : systèmes distribués (3 pts)

1. Que signifie, pour une structure de partitionnement, être *dynamique*. Avons-nous étudié un système où le partitionnement n'était pas dynamique ?
2. Quel est le principe de la reprise sur panne dans Spark ?
3. Un groupe de serveurs en mode maître-esclave comprend 5 nœuds. Suite à une fragmentation du réseau, le maître se retrouve avec un seul esclave. Expliquez ce qui se passe alors.